

编程启蒙：思维与代码

黄家宝 编著

北京航空航天大学出版社

内 容 简 介

本书以引导式教学为核心指导思想,以期为入门学习编程的读者提供更好的学习体验。

本书主要由“思维”和“代码”两篇组成。上篇“思维”是为了让没学过编程的读者了解编程究竟是做什么的,掌握认识和解决问题的思维方式;也能让有编程基础的读者更深刻地理解编程背后的思维方式,达到举一反三的学习效果。下篇“代码”是对 Python 代码的系统入门教学,不仅可以强化第一部分所讲的编程思维,而且能够帮助读者建立并培养属于自己的编程思维和逻辑思维。从变量、初步认识数据类型到函数,每一章都经过精心编排,结合引导式教学,逐步推进。例如,在初步认识数据类型章节,先大致介绍 Python 的各种数据类型,让读者了解各种数据类型之间的关系。因为学习任何新知识都无法完全孤立地进行,所以这一章的内容能让后续学习容易很多。

本书适合所有想要了解、学习编程的读者,其严谨的逻辑设计也使其适合专业计算机辅导机构和计算机专业师生阅读。本书的写作初衷是不为编程设门槛,这使得它适合各类想要学习编程的读者,让不同基础的读者各有所获,真正做到浅者觉得其浅,深者觉得其深。

图书在版编目(CIP)数据

编程启蒙:思维与代码/黄家宝编著. -- 北京:
北京航空航天大学出版社, 2025. 12. -- ISBN 978-7-
5124-4941-1

I. TP311.1

中国国家版本馆 CIP 数据核字第 202523UG49 号

版权所有,侵权必究。

编程启蒙:思维与代码

黄家宝 编著

策划编辑 杨晓方 责任编辑 张 凌

*

北京航空航天大学出版社出版发行

北京市海淀区学院路 37 号(邮编 100191) <https://press.buaa.edu.cn>

发行部电话:(010)82317024 传真:(010)82328026

读者信箱:tmp031256@buaa.edu.cn 邮购电话:(010)82316936

有限公司印装 各地书店经销

*

开本:710×1 000 1/16 印张:27.75 字数:624 千字

2026 年 3 月第 1 版 2026 年 3 月第 1 次印刷

ISBN 978-7-5124-4941-1 定价:99.00 元

若本书有倒页、脱页、缺页等印装质量问题,请与本社发行部联系调换。联系电话:(010)82317024

写给读者的话

“你好，我是悦创！”写下这个开头，我用了 8 年。我在各个平台的文章和自己的博客中都用这个开头，现在我仍在这本书中沿用它。

前言你可以不看，但是我必须写。不过我想在此告诉你，我的前言你还真应该读一读——因为它对你有益！

在写这本书的时候，我一度陷入了恐慌之中——害怕因创作周期过长，导致它无法正常出版。之所以耗时这么久，主要有以下几个原因：

第一，一直在反复打磨书稿内容，希望找到更好的表述方式，让新手和具有不同语言理解能力的人都能读懂它。

第二，我平时做的是海内外学员的私教，所以留给书稿编写的时间较少。

第三，从 2022 年 4 月 28 日签订出版合同至今（2025 年 4 月 26 日），已经过去了 1 094 天，也就是 2 年 11 个月 29 天，再过一天就整整 3 年了，这拖稿的确有点“过分”。

第四，在写这本书时，行业里大模型技术异军突起、迅猛发展，我一度陷入沉思：这本书还有存在的意义吗？

第五……

基于上面的各种原因，我一度担心本书无法出版……

不过，我还是很感谢跟我对接的杨晓方编辑，她始终耐心等待、从不催促，给了我足够的时间编写本书。正是因为她的这份耐心，才让这本拥有优质内容的书有机会与你相遇！

本书的缘起与内核

缘起·时代所需

我从小就喜欢文学，没想到长大以后，率先让我实现人生价值的却是编程。

我从某易离职之后，一直在当编程私教。我带过的学生遍布各行各业、各个专业，其中不乏名校学生。图 0-1 所示是我部分学生所在学校的名单。

所以当你翻开本书时，我想告诉你一个好消息：你和许多名校学生成了“同门”，因为你和他（她）们拥有同一位老师——就是我。

他们都来自哪些专业或行业呢？物理、化学、生物信息、英语、工程、计算机、数学、金融、会计、新媒体、证券等，都是他们所学的专业。从学员们的专业背景中，我深刻意识到：Python 编程正逐渐变成一项通用技能，一项人人必备的技能，而不再是特定职业人群的专属。

此外，经济发达国家的学生基本上在初中、高中、预科、本科、研究生阶段都可能开始学习编程。国内的少数学校也开始面向本科生开设 Python 编程课程，而大多数学



图 0-1 部分学生的所在学校名单

校则要求研究生学习编程。

当然,不论是国内还是国外,这一切的主导都是学生自己内心的想法,更准确地说,是思维是否觉醒,是否认识到编程在这个时代的重要性。否则,学生就会抱有“混”过这门课的想法。这会导致很多人在学习编程时只会机械记忆语法,却没有建立完整的编程逻辑,就像“盲人摸象只知其一”。这个编程逻辑并非仅适用于编程领域。希望你在阅读本书的“何为编程思维?”章节后能发现,编程思维是你在面对所有领域时都应该具备的思维方式。举个例子,请你看看下面这段代码,能否看懂它的大概意思——不需要完全理解或实现它。

```
x = 7
y = 3
z = x * y + 2
print(z)
```

零基础、没有任何编程经验的你,是不是也能轻松看懂上面的代码? 就算再复杂一些的代码,你也是可以看懂的。例如:

```
def add_up_to(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
result = add_up_to(5)
print(result)
```

就算你一点编程也不懂,也依然可以大概看懂它!这是为什么呢?因为 Python 语言很接近自然语言,所以你只要稍微懂点英语,就能看懂一些 Python 代码。所以很多人被学校或培训机构等忽悠——因为它们只一味讲语法。授课老师也不知道编程的核心是培养编程思维和建立完整的逻辑体系,以至于很多学生投入了大把时间,却依然无法独立编写代码。很多学生找到我的时候都会说:“老师,我懂 Python 语法,可就是写不出代码。”或者有学生说:“平时老师讲课,感觉也听懂了,可是写作业时就会卡壳,根本写不出来。”

对于如图 0-2 和图 0-3 所示的这种情况,除非学生自己只是想通过考试,但凡想真正掌握 Python 编程技术的人,都会选择进行系统的学习。

(所有聊天记录不涉及隐私,且当事人已知悉!)

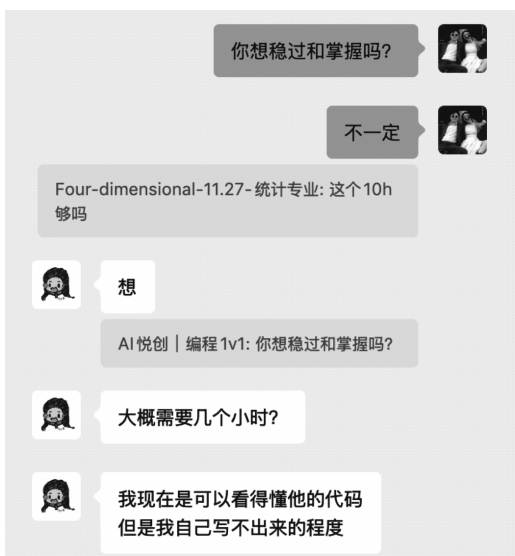


图 0-2 真实案例一

此时,他们就会陷入沉没成本的困境:

第一,从头学起,就得再次投入时间和金钱,之前的付出基本上就白费了。

第二,从头学起,就可能跟不上学校或工作中的进度。

只有用对方法,才能根治问题,真正节省时间。看似需要重新投入,实则是自我救赎。

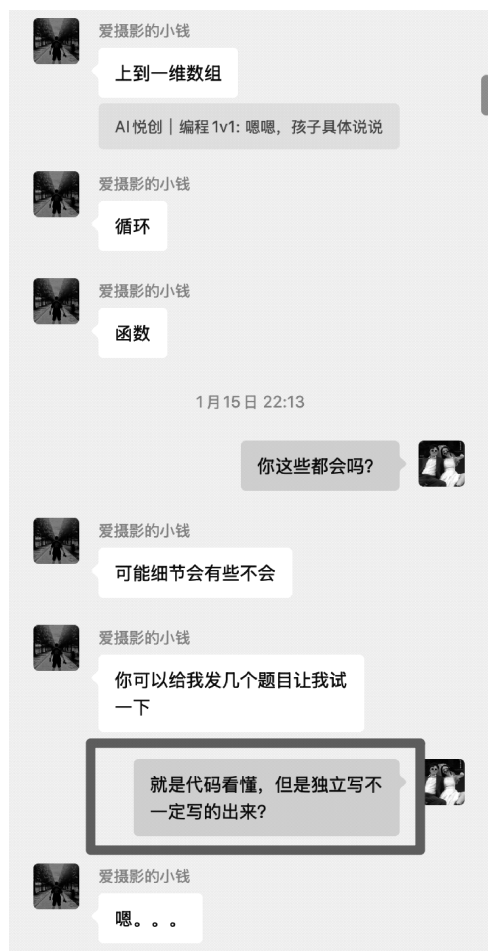


图 0-3 真实案例二

而根本性的解决方案则是:从头学习,在这个过程中要么自己研究、学习,要么有老师一步步引导着进行学习(引导式教学)。但现实中,要是能有其一,也不会让自己陷入当下的困境。

而跟随我学习呢,需要支付一定的费用。然而,费用还不是主要的——只要你有觉醒的意识,这一切就都好解决;更重要的是,我每个月都是根据我的实际精力在保证教学质量的情况下来招收私教学员的。

我写这本书的真实意图之一是将那些原本只面向精英人群的优质学习资源,以书籍的形式展现给你。

另外,我想补充一点:不仅仅是学生,已经工作的人也可以找我学习编程,如图 0-4 所示。难道是为了转行吗?——不是的,不是因为工作要求,而是因为编程可以极大地提高工作效率。

还有些学生早期没学过,等到在学校学习的时候,老师不会从零开始一点点地去



图 0-4 职场中的真实需求例子之一

教,公司招聘时也不会给你培训,而是会直接招收已经学会的学生。

图 0-5 所示的聊天记录截图也是这类案例之一,因为在国内的高中没学习过,所以无法及时跟上国外学校的课程进度。



图 0-5 统计学·必修 CS

由此我希望你可以 Get 到如下 Point:

要点一:你有幸成为名校学生或各职业领域精英的同学,因为你跟他们拥有同一位老师。

要点二:认识到学习编程的必要性,它不是可选项。

要点三:学习编程的本质,是需要有意识地建立起编程逻辑。

要点四:不要等到需要用的时候才想起来去学。对于有些知识,只有你提前学了,在需要的时候才能游刃有余。

要点五:我们学习某项技能,有时候不仅是为了自己在将来的工作和学习中去使用它,更是为将来解决问题储备一种思路。例如,使用 Python 可以批量收发邮件,但是不熟悉、没有学过 Python 的人是不会想到使用 Python 去批量收发邮件的,而了解、学习过后,就算自己不会,也可以找人帮忙实现。

要点六:本书将贴切地剖析 Python 入门的难点,使用引导式教学的方式进行阐述。

要点七:人工智能技术正在爆发式发展,想要更好地使用人工智能和 AI 工具,就必须学会 Python 编程,做大模型的主人,融入这一科技跃迁的进程中,而不是只做一个“历史的旁观者”。

补充:重学不是单纯的重学,主要的优先级排序是思维逻辑>笔记>语法。对于需要考试的人来说,语法需要记忆。对于不需要考试的人来说,语法记不牢也没关系,要学会查找自己的笔记,毕竟实际应用并不是“闭卷”。不论是哪类人群,都要按照这个优先级来进行学习,做到熟能生巧,这样,就算记不住语法,用多了自然也就记住了。

注意:要学会记笔记,不论是线上私教学员,还是本书的读者,都要养成做笔记的习惯。只有自己做笔记,自己才能熟练使用。并且笔记要做云端的电子版的,以便随时随地查阅。学习编程时记笔记的方法与以往学习其他学科时的手写笔记不同,编程笔记要考虑查找速度和便捷性,因此不要选择手写笔记。

对于上面讲到的要点五,我要做一些补充:

在学习一项新技能时,最重要的收获往往不是马上就能把它用在工作或学习中,而是当你遇到问题时,脑海里会多一个“可以解决它”的选项。例如,在没学过 Python 之前,如果遇到一堆数据需要处理,比如要删掉文本中的所有数字,你可能只会一味地手动操作:一个个选中、一个个删除,甚至心里会想:“没有办法,只能慢慢来吧。”但当你学了 Python,甚至稍微了解了正则表达式之后,你会突然意识到:原来只要一行代码就可以自动把所有数据清理干净。

这不是简单的“学了之后有多快”的事,而是学了之后你能想到还有另一条路,而没学的人,根本不会意识到还有这条路。

所以,学习的根本意义是:使你在未来遇到问题时,能想到更多可能性。这才是掌握技能带来的真正自由!

本书要带你学的不仅仅是编程思维、Python 入门知识等,更重要的是帮你培养一种解决问题的思维方式!

最后再补充一句:2025 年教育部发布了各大高校新增的 29 个专业,如图 0-6 所示。你可以看看,虽然类别不同,但是有多少是与技术有关的呢?都是哪些大学呢?这些新专业的相关课程将在部分重点大学进行试点,然后逐步推广至更多院校。

序号	专业名称	专业代码	所属专业类	学位授予门类	布点高校
1	人工智能教育	040117TK	教育学类	教育学	北京师范大学
2	婴幼儿发展与健康管理	040118T	教育学类	教育学 管理学	陕西学前师范学院 山东女子学院
3	航空运动	040217TK	体育学类	教育学	吉林体育学院
4	区域国别学	0502104TK	外国语言文学类	文学	北京外国语大学
5	海洋科学与技术	070705T	海洋科学类	理学	同济大学
6	智能视听工程	080722T	电子信息类	工学	中国传媒大学、山西传媒学院 南京传媒学院、西安邮电大学
7	工业软件	080919T	计算机类	工学	东莞理工学院、广东工业大学
8	时空信息工程	081206TK	测绘类	工学	武汉大学
9	智能分子工程	081309T	化工与制药类	工学	大连理工大学
10	生物质技术与工程	081707T	轻工类	工学	四川大学
11	智慧应急	082905TK	安全科学与工程类	工学	防灾科技学院
12	低空技术与工程	083203TK	交叉工程类	工学	北京邮电大学、华南理工大学 西北工业大学、北京航空航天大学 南京航空航天大学、北京理工大学
13	集成电路科学与工程	083204TK	交叉工程类	工学	复旦大学、重庆邮电大学、 南京邮电大学
14	碳中和科学与工程	083205T	交叉工程类	工学	北京师范大学、北京科技大学、 昆明理工大学
15	智慧城市与空间规划	083206T	交叉工程类	工学	武汉大学

序号	专业名称	专业代码	所属专业类	学位授予门类	布点高校
16	药物经济与管理	100709TK	药理学类	管理学	中国药科大学、温州医科大学 山东中医药大学
17	医疗器械与装备工程	101015TK	医学技术类	工学	徐州医科大学、山东第一医科大学
18	健康与医疗保障	101016TK	医学技术类	管理学	黑龙江中医药大学、重庆医科大学
19	老年医学与健康	101017TK	医学技术类	管理学	南昌大学、宁夏医科大学 昆明医科大学
20	人才发展与管理	120421TK	公共管理类	管理学	广东财经大学
21	国际邮轮管理	120905TK	旅游管理类	管理学	上海海事大学
22	舞蹈治疗	130214TK	音乐与舞蹈学类	艺术学	南京特殊教育师范学院
23	音乐科技	130215T	音乐与舞蹈学类	艺术学	南京艺术学院
24	数字戏剧	130316T	戏剧与影视学类	艺术学	中央戏剧学院
25	数字演艺设计	130317T	戏剧与影视学类	艺术学	上海戏剧学院
26	智能影像艺术	130318T	戏剧与影视学类	艺术学	中国传媒大学
27	虚拟空间艺术	130414T	美术学类	艺术学	北京电影学院
28	人居设计	130514T	设计学类	艺术学	上海交通大学
29	游戏艺术设计	130515TK	设计学类	艺术学	中国传媒大学、北京电影学院 山东工艺美术学院

图 0-6 教育部高校新增专业

别想着被别人拯救,真正可以拯救你的只有你自己。拿起本书学起来吧,蜕变从此开始。

内核·为什么本书值得学习?

编程这个学科交叉性极强,不像传统的数学那般:先学习加减乘除,然后反复练习加减乘除。换句话说:对于“先学知识,再用题海战术”的学习模式,我既不提倡,也不建议!为什么呢?假如编程目前就给你讲了变量、数值类型(包含运算符),那要给你留什么作业呢?留下创建变量、练习加减乘除这类简单操作的作业吗?这些练习很简单,无须反复进行,甚至可以说反复练习是无效的。

我在写本书时,有以下几个核心考量:

核心一:写作总耗时三年有余,内容不断地进行打磨

这本书从动笔到定稿,经历了三年的时间。在这三年里,我不断地推敲和完善内容,力求让每一个知识点都表达得更加清晰,讲解得更加自然。我不急于求成,只希望通过长期的打磨,让读者在阅读时能感受到思路的流畅感和内容的扎实。

核心二:在教学实践中不断修正

在这三年里,我在写本书时也在不停地使用书中的内容授课,一旦发现其中有问题,我就马上去解决。所以,本书的内容在你阅读之前就已经被我实践过了!(省字版说法:各个行业、各个专业的学生都学习过,教学经过了验证)

因此,读者在学习过程中遇到的问题,很多都已经在实际课堂上提前被发现并解决了。(如果没有,请随时向我反馈。)

核心三:采用引导式教学方式,真正为新手考虑

本书采用引导式教学思路编写,尤其是在编程语法部分,不急于追求篇幅上的简短,而是更注重过程的引导与细节的讲解。考虑到初学者在起步阶段容易遇到各种障碍,我选择用朴实、耐心的表达方式,以帮助读者循序渐进地掌握每一个关键知识点。

核心四:内容体系化、结构化,清晰易学

全书从内容安排到学习节奏,都严格遵循体系化、结构化的原则。每一章、每一个知识点,都精心编排,循序渐进,层层递进。学习的过程不会跳跃,也不会杂乱无章,而是像搭建积木一样,一步步地夯实基础,帮助你稳稳地走上编程之路。

核心五:紧跟最新前沿,教授原则性知识

很多东西很容易过时,比如,“经验是会随着时间过时的,而很多书籍教授的却只是经验而不是原则。”这么说有些抽象,换句话说:“很多教学类书籍只是单纯地教授知识,而不教授如何获取知识、如何思考、如何从零开始一步步地推导,这些都被大部分作者省略了。”为什么呢?无非有两个原因:

原因一:部分作者自身还未完全理解就开始写书,因此只能重复已有的知识,而不能带领读者看透本质、深入学习。面对读者的提问,这类作者的回答也略显生疏。

原因二:原则性的教学需要耗费大量心血。很多写教学类图书的作者是为了名气、为了“割韭菜”而写书,而不是为了学术追求,也不是自身达到某种水平后,出于内在自驱想要分享经验。这类作者往往不会细致地去写,不会去考究书籍的逻辑性和科学性。

对于原因二,当下有太多例子,我就举一个例子:DeepSeek 刚问世就迅速走红时,某位作者的相关书籍就出版了。他预料到 DeepSeek 会火,所以提前写,提前交稿给出版社。

如同我前面所说:“别想着被别人拯救,真正可以拯救自己的只有自己。自己脚踏实地地去学才是王道。”所以,不要以为只要报班学习某视频博主的课程,就可以跟上 AI 时代了。只有自己脚踏实地地学习,才能真正跟上 AI 时代。

在看到 DeepSeek 相关图书问世的时候,我更加意识到,国内需要我所写的这类书籍。或许我不是最好的,但我愿做先行者。

对于原则的理解,我觉得没有什么比《羊皮卷》一书中所讲的更到位了:“我不会让所谓的知识或者经验妨碍我的行程。造物主已经赐予我足够的知识和本能,这份天赋是其他生物望尘莫及的。经验的价值往往被高估了,人老的时候开口讲的多是糊涂话。说实在的,经验确实能教给我们很多东西,但往往需要花费太长的时间。等到人们获得智慧的时候,其价值已随着时间的流逝而减少了。结果往往是这样,经验丰富了,人也

余生无多了。经验就像时尚,适合某一时代的行为,并不意味着在今天仍然可行。”“只有原则是持久的,而我现在拥有了这些原则。这些可以指引我走向成功的原则全部被收录在这几张羊皮卷里。它教我如何避免失败,而不只是获得成功,因为成功更是一种精神状态。人们对于成功的定义,见仁见智,而失败却往往只有一种解释:失败就是一个人没能达到他的人生目标,不论这些目标是什么。”

核心六:避免图书一出版即“过时”

很多技术类书籍出版后就“过时了”,但是本书不会,原因有二:首先,本书教授的大部分是思维和原则,它们是亘古不变的;其次,我提供了配套网站的链接,方便对内容进行实时维护和补充。

与文学创作或其他科普、教学类图书不一样,要想出一本聚焦思维与编程的好书极其耗费时间。这本书或许不是最好的,但肯定是开山立派之作。

浅谈(有效信息)

你学到手的有效信息有多少?换句话说:你在听一个人上课或演讲时,能接收到多少有效信息?如果感觉表达不太到位,就应该这么说:老师上课时传达给你的知识,到你那里还剩下多少?举个例子:我曾给伦敦政治经济学院毕业的吴同学上课,为她讲了一系列 Markdown 操作,语速适中,还一步步带着她实操并拆解讲解。等到吴同学自己操作时,她总是会少那么一两步。例如,我讲了 1、2、3 步,吴同学会操作 1、3 步或者 1、2 步,总之就是不能完整复现。

要想高效地得到完整有效的信息有两个前提:

前提一:老师要表达清晰,语速不能过快,要结合学生的理解能力;

前提二:学生自身的独立思考、理解能力需要强,否则前面说后面就忘了。

那吴同学卡在哪里了?——前提二。

没有很好的实时理解能力,也就是对新信息的实时处理能力较弱,但如果是一对一教学,最后都能成功掌握。

我想要表达的意思很简单:上私教课的学生,不论理解能力如何,都可以通过视频课程的形式学会;但是,一些未上私教课的学生,却会因为未被老师单独指导而无法掌握部分知识。而书籍就能很好地解决知识单向线性传递的局限,因为文字一直都在,不会消失,学生可以反复阅读、巩固学习。所以,通过本书来学习,其效果并不会比实时上课的效果差太多。

江湖地位

很多人写书是为了提升名气、满足学术指标,或者像一些短视频博主那样,趁着大模型的热度割一拨儿“韭菜”,匆匆出书。但比起这些,我更在意的是一种“江湖地位”。如果有一天,人们想要学习编程时会自然想起我写的这本书——那就足够了。

教育理念

怎么才能教得好？

做私教 8 年以来，很多人都会问我：“你怎么教得那么好？”我说：“不是我教得好，而是我学得好。”因为只有学得好，只有对自己研究的领域有一定的体会，才能在讲课时表达清楚。

教学相长，不停授课的过程也是不断学习、收获的过程——教学本身就是一个持续学习的过程。

此时，我写书只是为了梳理逻辑，把过往的生活重新过一遍。

感悟：书籍的作用是什么？

很多人误以为书籍不过是信息的一种载体，是知识的一种集合。但我渐渐明白，书籍真正的作用远远不止于此。我的感悟是：书籍并不是简单地把知识集结起来出版、收费就可以了。书籍其实是将原来珍贵的知识借助自身这个载体来传递，使其变得便宜了。

书籍是土壤，知识是在这土壤上开出的花、结出的果。书籍可以帮我们做什么？

书籍也没办法帮助把那些原来没有价值的知识变得更有价值。书籍可以做的事情是：为知识带来一种类似互联网的规模效应——那些原本难以触达大众的知识，得以低成本、大范围传播。这是一个经济学原理。举一个例子：我在做线上编程一对一教学时帮助过很多学生和职场人士。每天会接触大量不同类型的学生，也有很多创业者来找我，我很愿意帮助大家。1~30 个人来找我，没有问题，可是 40 个人来找我，就有挑战了，100 个人、1 000 个人……我没办法服务这么多人，我在工作中需要不断地跟大家说“对不起，对不起，我没有时间”。为什么呢？因为时间是我一对一教授知识的载体，但同时也给了我一个上限，导致我没办法在“单位时间”内接触那么多真正需要帮助的学员——也就是那些打算用技术辅助工作和学业的人。并且，想要跟着我上课，需要支付适当的学费，所以很多普通家庭或者不重视编程的家庭也不会投入太多——虽然编程很重要。而通过书籍学习，只需要付一本书的费用即可。比如想要听罗翔的法学课，你可能需要考 600 多分、考入中国政法大学才能现场听课，但通过他的书籍，只需花一本书的钱就能学习。

所以，那些优秀的老师，他们一辈子服务的学生可能不超过几千人，只能惠及少数人，但是书籍却能帮助我们传播“最优秀”“最珍贵”的知识，让所有人都有机会学习。所以，优秀的作家们正在做一件非常了不起的事情——通过书籍使最优秀的资源得到最广泛的传播。换句话说就是：书籍并不是简单地把信息从作者那里传递到读者那里，它的意义在于：把原本分散的、昂贵的，甚至难以直接接触到的智慧，经过整理、提炼和沉淀之后，用一种可复制、可流通、可负担的方式，传递给更广大的人群。

书籍的出现，本质上降低了知识传播的边际成本。原本只有少数人才能接触到的经验、思想和发现，通过印刷出版，通过流通，可以让更多人以低成本获取。这是规模效

应的一种体现,也是一种能展现社会进步的典型现象。换句话说,书籍并不会凭空创造价值,但它极大地放大了原本就存在的智慧,让每一个渴望成长的人都能以极低的门槛触摸到人类最璀璨的思考成果。这就是书籍真正的意义。

浅者觉得其浅,深者觉得其深

什么是最优秀的资源?

图书出版在给我带来喜悦的同时也给我带来了一个挑战:当我出版的图书流向 10 多万人手中的时候,这 10 多万人的地域分布和身份背景就变得非常广泛了。如何给 10 多万人讲好编程课?以我自身的私教课程为例,学员遍布全球,有些是我的同行,是计算机程序员,有些是学生(各种专业)和上班族,还有一些是 80 多岁的老人(退休教师),甚至还有一些四五岁的幼儿。要面对这么多差异性明显的学员,我应该怎么讲这个课?应该讲得更深吗?那样的话,可能很多人没办法理解。应该讲得更浅吗?那些已经有丰富工作经验或已经“学过”一遍的人,可能又会觉得这样的内容没有价值。

其实这个问题在一对一教学中很好解决,只需结合学员的情况授课即可。但是,现在这个问题在图书中出现了,我如何才能面对 10 多万人把编程讲好,这对我来说是非常大的挑战。

到底怎么才能讲好?我做私教这 8 年学到了一件非常重要的事,就是真正好的老师,不应该让浅者觉得其深,也不应该让深者觉得其浅,好的老师应该是让浅者觉得其浅,深者觉得其深。这是什么意思?就是要讲透,最终要达到的效果是:

第一,那些没有接触过编程的人读完之后会觉得编程其实很简单,既好玩有趣,又没有想象中那么复杂。

第二,让那些自以为会编程的人阅读完之后惊出一身冷汗——原来这才是真正的编程。

第三,换句话说:初学者在学会了我所讲的东西后,会觉得原来这些知识挺简单的,并不像之前想象中的那么难。而这些方面的深入学习者在学习我所写的书时,会觉得犹如醍醐灌顶,惊叹:“原来这个知识还能这样理解啊!”

第四,深者觉得深,浅者觉得浅——就是一个字叫“透”,只有理解透它,才能讲透。

第五,额外说一句:如果你问一个人问题,这个人回答:“你将来达到某种水平后就能理解了。”这其实就是这个人本身理解得不够透彻导致无法讲透的表现。

怎么才能讲透?我找到了榜样——几位奥数老师。

我的女儿名叫棠棠。未来我准备让她学习奥数,这些老师正是我要找的。

我出个题目:如何跟小学生讲明白“鸡兔同笼”的问题?这是考验你能不能当好老师的一个非常重要的标准。在一个笼子里,鸡和兔加在一起有 35 个头、94 只脚,请问这个笼子里面有多少只鸡,多少只兔子?对这个问题的不同层级的分析如表 0-1 所列。

我想读者们都会解这道题,因为大家都学过二元一次方程组,这道题对你们来说太简单了,只需要列出方程组,鸡和兔加在一起共 35 个头,一只鸡有两只脚,一只兔子有四只脚,加在一起共 94 只脚。

表 0-1 不同层级的分析

深	透
(列方程组) 鸡 + 兔 = 35 $2 \times \text{鸡} + 4 \times \text{兔} = 94$	鸡有 1 个头, 2 只脚 兔有 2 个头, 4 只脚 请兔子站起来, 2 脚变 2 手, 现在鸡兔共有多少脚? $2 \times 35 = 70$ 只脚
(变形) $2 \times \text{鸡} + 2 \times \text{兔} = 2 \times 35 = 70$ ① $2 \times \text{鸡} + 4 \times \text{兔} = 94$ ②	但题目说一共有 94 只脚, 多出的: $94 - 70 = 24$ 这 24 只脚是谁的? —— 是兔子的“手”。
(② - ①) $2 \times \text{兔} = 94 - 70 \rightarrow \text{兔子} = 12$	每只兔子有 2 只手: $24 \div 2 = 12$ 所以, 兔子有 12 只, 鸡有 23 只

此外,我们都学过怎么解这类方程组,将方程 ① $\times 2$,再用方程②减去这个结果,就解出来了:鸡有 23 只,兔子有 12 只。

你们都会做这个题目吧,但是现在探讨的不是怎么解出来,而是如何向小学生讲明白怎么做,以及小学生听完之后有什么感受。要知道,他们根本不懂什么叫方程组。

要向小学生讲明白这件事,该怎么讲?

鸡有两只脚,兔有四只脚,真的是这样吗? 不对,兔子有两只脚,另外两只是手,鸡和兔都是两只脚,一共 35 个头,都有两只脚,加在一起是多少只脚? 70。简单吧? 你说 94 里剩下的 24 只是手,那是多少只兔子的手呢? 一只兔子两只手,24 除以 2,是 12 只兔子。这样,小学生是不是就听明白了?

但是你让大学生来听,他们可能会惊出一身冷汗,原来自己以前并不知道这道题还可以这么解,更没意识到这背后藏着对题目更深刻的领悟。

所以,一个优秀的老师必须透彻地理解教学内容,必须讲自己最宝贵的东西,真的对这一行有非常深刻的理解。

我上过小学,你也上过小学,讲给小学生听有什么问题呢? 事实上,如今只有真正的大师才能讲好小学知识。这是我做私教这些年学到的非常重要的一点,就是让浅者觉得其浅,让深者觉得其深。

所以,每一位知识服务者都应该是这个行业里的大师。所谓“透”,不是把复杂讲复杂,而是把复杂还原为常识。当一个问题连小学生都能理解时,它往往才真正被理解。

这些年的心得(思想·法则)

此部分有点像“名言警句”,可以进行选择性阅读。但是我推荐你阅读一下,并在平日的学习中思考这些心得。

手札 1:我们有时候不仅仅要看选择项以内的答案,也要去思考选择项以外的答案。——黄家宝 2022 年 5 月 16 日

手札 2:每个人用 AI 都是千篇一律地直接问,如果每个人的问题相似并且用的是

同一个大模型的话,那么 AI 回答的结果基本上是一样的,不会有太大的变化。真正不一样的核心就是我们赋予它一个核心思想或核心点,这样才能释放 AI 的真正生产力。AI 再厉害,也得有人给它核心! ——2025 年 4 月 28 日 23:51:04

手札 3:在学习编程时,往往有学生习惯性地仅凭想象去判断代码某一阶段的输出结果,从而无法进行下一步编写。此时,通过适当的输出(`print()`)可以减轻学习压力。——2025 年 4 月 28 日 23:52:46

手札 4:先实现功能,再去优化,否则一切会很乱。——AI 悦创 2022 年 6 月 2 日

手札 5:凡是你不能清晰写下来的东西,都是你还没有真正理解的东西。——2023 年 1 月 29 日 14:40:30

手札 6:有些事情很遥远,你争取的话,它就离你很近了! ——2023 年 1 月 30 日 11:23:35

手札 7:语言的高贵,不是取决于它的长短,而是取决于作者的心意。——与 FM 交流 2023 年 2 月 16 日 00:05:20

手札 8:不仅要纯粹地思考,还要动手验证。——给高天琦上课 2023 年 2 月 26 日 20:27:04

手札 9:编程应该是一种思维、一种创造、一种乐趣,而不是一种负担,这才是它本该有的样子。——与 Jason 上课后 2023 年 7 月 7 日 23:04:10

手札 10:当一件事情超出你的认知时,你就会变得小心翼翼,生怕生出变故。比如,我花钱找人调试麦克风时,工作人员给我安装了一堆我不认识的软件,于是我当时直接问:“我之后升级系统会不会受影响?”她说不会。但我内心并没有平静,因为我并不知道我担心的事会不会真的发生,以及我该怎么解决我有可能遇见的问题。(虽然最后没有调试好,我还是申请了退款。)——收拾桌面时想到了麦克风 2023 年 7 月 25 日 13:53:47

手札 11:群里没有大佬,我只是比你们早接触、早研究而已,真正的大佬是开发这些开源工具、模型等的人,愿我们都虚心地学习。——认知①2023 年 7 月 27 日 14:09:12

手札 12:普通人如何在 AI 时代赚钱? 别异想天开了,你想这个时候,你要么已经成为别人的“韭菜”,要么就是在成为“韭菜”的路上。现在学编程是给未来增强竞争力,想直接变现几乎是不可能的。

学编程有用吗? 有,只要你是想融入你自己的领域或者是为未来做准备就有用。但是,如果你想变现,那是非常困难的。——认知②2023 年 7 月 27 日 14:10:44

手札 13:国外是看家庭情况卷,国内不看家庭情况卷。——张扬 2023 年 10 月 24 日 21:31:14

手札 14:现阶段的 AI 仍然需要人工去补充与完善。——AI 悦创 2023 年 11 月 10 日 11:24:19

手札 15:做任何事情的时候,如果感觉到麻烦,千万别犯懒,也别烦躁。此时应多想一步:有没有更简便的方法? 因为这正是内驱力(内在动力)推动你精进学习的时机。——2023 年 12 月 30 日 11:24:34

手札 16:有时候,不要只看局部。站得高点、站得远点,我们就能拥有全局视角,这样往往更便于发现事物背后的关联或规律。——AI 悦创 2024 年 1 月 1 日 23:17:10

手札 17:爱不是某一次的集中表达,而是散落在日常中的平凡表达。——AI 悦创 2024 年 4 月 3 日 16:10

手札 18:我发现所有的称呼都无法彻底表达出我对你的深情。——2024 年 3 月 8 日 20:09:15

手札 19:在现实中你做到了,那代码就是你现实行为的映射。——2024 年 6 月 28 日 22:43:02

手札 20:移除重复字母并保持顺序,如图 0-7 所示。编写一个函数 `remove_duplicates(s)`,它接收一个字符串 `s` 作为参数,返回一个新的字符串。该字符串由原字符串 `s` 中的字符组成,每个字符只出现一次,并保持它们在原字符串 `s` 中的顺序。例如,`remove_duplicates("banana")` 应返回“ban”。

你以为的结束,没准只是开始。——2024 年 6 月 28 日 23:25:14

```
1 """
2 Student Code here
3 def remove_duplicates(s):
4     index_s = 0
5     lst = []
6     for i in s:
7         if s[index_s] in lst:
8             lst.append(index_s)
9         elif s[index_s] not in lst:
10             index_s += 1
11     return lst
12
13
14 result = remove_duplicates("apple")
15 print(result)
16 """
17
18 def remove_duplicates(s):
19     index_s = 0
20     lst = [] # 初始化是空列表
21     for i in s:
22         if s[index_s] in lst: # 而你使用 s[index_s] in lst, 这个是恒为 False。永远不在列表里面。
23             lst.append(index_s)
24         elif s[index_s] not in lst:
25             index_s += 1
26     return lst
27
28
29 result = remove_duplicates("apple")
30 print(result)
31 """
32 def remove_duplicates(s):
33     index_s = 0
34     lst = [] # 初始化是空列表
35     for i in s:
36         if s[index_s] not in lst: # 而你使用 s[index_s] in lst, 这个是恒为 False。永远不在列表里面。
37             lst.append(s[index_s])
38             index_s += 1
39
40         elif s[index_s] in lst:
41             index_s += 1
42     return lst
43
44
45 result = remove_duplicates("apple")
46 print(result)
47 # 你以为的结束,没准只是开始~2024-06-28 23:25:14 题目要的是得到字符串
```

图 0-7 函数代码编写

手札 21:我给你讲解的代码示例,是我认为你掌握这个知识点必不可少的演示环节,而你在理解这个示例的过程中遇到的问题,才是真正值得你深入学习的地方。——2025 年 2 月 28 日 09:06:05 学习思维

手札 22:在学习编程的过程中,看不出代码是否有语法错误与不会使用代码是两码事!我已经给你抄正确代码了,你还不能找出自己抄写代码时存在的问题,那请不要把问题归咎于“不会”。“不会”可不背这个锅。——2024 年 9 月 29 日

手札 23:只有理解了过程,才能开展研究、进行创造。读书时被动地先了解,吸收信息作为思考的源泉。但推导能看见树木,思考能看见整个森林,要把搞清楚的权利交给自己。(张朝阳真厉害,没想到!理科学霸对于读书也有独到的见解!他推荐的“1/3 读书法”效率特别高——即 1/3 的时间思考、1/3 的时间读书、1/3 的时间推导。这是什么意思呢?就是在读书前先明确自己为什么要读这本书,想从其中获得什么信息或知识。带着具体的问题去读书,最后再去验证推导。)——2024 年 10 月 10 日

手札 24:AI 可以帮我们处理很多事情,比如批量把类似“2025-02-28 09:06:05”格式的时间转换成“年-月-日”格式。但是如果数据重要,你还会交给 AI 操作吗?所以,有时候很重要的数据还是需要自己去处理,而不是交给 AI。——2025 年 4 月 29 日 07:03:36

手札 25:恩格尔系数吃饭占比越少表明越有钱的本质是:吃饭肯定是要花钱的,所以越有钱就越会在其他方面花钱,因为只有当物质富足后,才会开始追求精神层次的富足。所以除了食物的开支,其他的开支越多,越会间接导致食物开支占比减少。但食物开支真的变少了吗?并没有,只是被其他开支“分担”了!——2024 年 10 月 15 日

手札 26:有人学得快,有人学得慢,这既与学习方法有关系,也与思维方式有关系!——2024 年 10 月 15 日

手札 27:在 2024 年外滩大会上,“互联网教父”凯文·凯利谈到了 AI。他认为,在 AI 时代,答案变得唾手可得,最有价值的是提出正确问题的能力。如果你是毕业生,那“学会如何学习”就是你必须掌握的核心技能,因为你两年后的工作岗位,现在有可能还没有被创造出来。——2024 年 10 月 16 日

手札 28:题目:已经有了判断一个键是否在字典中的方法,那如何判断一个值在不在字典中呢?问题转换思想:当我们要判断一个值在不在字典中的时候,调用 `values()` 方法,问题就会变成:判断一个值在不在列表中。用已有的知识解决新问题,才是我们要学习的。——2024 年 11 月 29 日

手札 29:为什么笔记需要有大纲?没有大纲意味着找内容需要从头看到尾并且不能跳过。为什么不能跳过?因为你不知道你需要的内容在哪个区域,就只能从头看到尾地查找。这恰恰表明大纲的重要性,有大纲就可以快速定位自己所需要的内容在哪个部分。——2025 年 3 月 5 日

手札 30:有时候,一段代码写不写得出来不重要,重要的是你是否进行了有效思考。这样在我讲解的时候,你的思维就会跟着我的讲解走。反之,如果直接听讲解,你可能能够直接听懂内容,但是思维无法同步,也就无法真正掌握。——2025 年 2 月 26 日

手札 31:教育不是一味地输出,而应该把找答案的过程还给孩子。——2025 年 3 月 10 日

手札 32:读大学时,我的很多舍友都在玩电脑游戏,我却一直认真学习。他们现在“混得并不好”,但不是玩游戏的缘故,而是没有合理分配学习和游戏的时间。例如:柯洁也会玩游戏,但围棋还是会一直练习的。所以,游戏不是原罪,你那颗不会控制的心才是。——2025 年 3 月 18 日

手札 33:学习软件,未来却不干软件这行,那岂不是白学了?这时我就想到蒙曼老师的话:“读书记不住没关系,因为读书跟吃饭喝水一样,它是一个长期的过程,那水的作用是什么呢?是滋养我们的身体,每一滴水都是有价值的。读书的作用是什么?滋养我们的心灵(灵魂)。你读的每一本书,其实都会沉淀为你的审美观、道德观、价值观,这些才是对人至关重要的东西。”未来是一个科技时代,你会一点、懂一点计算机就可以融入这个时代,而不是旁观这个时代。——2025 年 3 月 18 日

手札 34:做研究的时候要控制单一变量,这样才能获得更可靠的实验结果。“例如:你要研究吃饭对身体的影响,就要控制其他因素,例如睡眠、运动等。只有当吃饭是唯一变量时,你才能更好地研究吃饭对身体的影响。”如果实验中的变量太多,你就无法知道是哪个变量导致了结果。——2025 年 3 月 30 日

手札 35:上大学的途径主要有两种:特招、高考。如果小悦被特招进清华大学,清华大学还有必要看小悦的高考成绩吗?——生活是代码,代码也是生活。程序员是人和机器之间的桥梁。——2025 年 3 月 26 日

手札 36:教育不是你找 100 个教育专家聊就能解决你孩子的学习问题,而是聊了之后受到了启发且要去行动,否则一切都是空谈。很多人找老师聊天,只是在潜意识里以为这样就可以缓解焦虑,但实质上是无法缓解的!——2025 年 3 月 20 日

手札 37:在记笔记时,不要试图擦掉笔记中的错误,而要保留这些错误,并且在附近记上正确的内容。错误是我们学习的见证,是我们来时的路!(因为这样会使笔记变得不真实。在笔记的下方或者旁边写上正确的内容,才能让笔记更具实用性,更加有效果。不要试图“擦掉”自己以往学习中遗留的错误。)——2025 年 3 月 26 日

手札 38:不要以为绘本只是给小孩看的,好的绘本其实也是适合给大人看的。因为好的绘本其实是有深度的,甚至比一些成人书籍还要有深度。家长看了还能在不知不觉中给小孩言传身教……——2025 年 3 月 28 日

手札 39:很多人在学习编程的过程中,经常因为怕麻烦、懒得备份而不去删除多余的代码,导致代码不能高效运行。换句话说:明明要看、思考的是程序 2,但是运行程序 1 影响着你观察程序 2,这样就在无形中增加了干扰因素。(省略版说法:多余的代码要去掉,否则会增加你的学习压力。)——2025 年 3 月 27 日

手札 40:人工智能是没有创意的,特别是在写书稿的时候。只有当人提供创意和灵感方向时,才有可能让人工智能完善。当然,人工智能偶尔也能给出一些平时没注意到的点。总归还是有点用处的吧……谁知道呢。——2025 年 3 月 29 日

手札 41:写代码的正确原则:原则 1:在理清实现逻辑,再开始写代码;原则 2:如果

无法一次性想好完整的实现逻辑,不必等全部想清楚再动笔,可以先把当前已经明确的部分写出来,然后在实践中继续思考和推进,以此持续激发大脑思考。——2025年3月29日

手机 42:在平时学习编写代码时需要注意什么?要避免编写完代码就直接运行得出答案,应该先用大脑思考、推导出结果,再运行代码验证。要对答案和结果保有期待,不要让自己失去真正有效学习与思考的机会。——2024年11月12日(视频版:<https://www.bilibili.com/video/BV13bm4YREh5>)

重点:是否需要频繁练习?

对于这个问题,仁者见仁,智者见智。

每当我给私教学生讲解到某个章节(例如变量、字符串、列表、数字型等)时,学生都忍不住问我一个问题:“老师,这次下课后,有什么作业或者练习让我练习一下吗?我想巩固一下。”其实我心里明白,学生问这个问题,是因为他们想要一个明确的练习目标,而不是单纯听课。其实我也很想给学生布置作业,但编程这门学科有其特殊性——知识点交叉性太强,随便一道题都可能涉及后续还没学的语法。这也是很多学生无法真正学好编程的原因,很多老师不会考虑“交叉性”问题,只是图省事直接出题,不考虑学生是否学过,导致只有少部分学生会做,而大部分学生被迫放弃。而前面的语法,比如变量、数字型,这些讲完之后,其实也出不了什么新颖的题目。总不能让你们反复创建不同的变量,或者反复编写加减乘除的代码吧?这些你们本来就会,没有必要再去练习,这样的行为只是假性练习,没有意义。并且这种练习只是练习熟练度,没有必要。如果还是为此出题,其实是为赋新词强说愁!举个例子:

下面我用一个你我都熟知的数学问题来说明。先问你个问题:“国外小学生用计算器是因为数学不好吗?”

你可以先想想,然后再继续阅读。

我的解答是:这件事只是我们教学的重点不同,我们中国的小学很多时候是在训练孩子们的熟练程度。

国内小学生会练习大量列竖式计算的方法:加法、减法、乘法、除法,如果换成国外小学生做这类计算,会怎么样呢?——根本都不会,他们进行数学计算都会直接用计算器。因为国外小学生压根没学过“竖式”这种计算方法,他们的教学重点仅仅是理解加减乘除的本质,不关心对加减乘除计算的熟练程度,也不会针对熟练程度进行过多的训练。

此时,你可能会问我哪种方式更好。其实,我个人觉得,国外的这种方式或许更好一些。因为加减乘除类计算能力,在有了计算器之后,其实都没有太大的作用了。我们真正需要训练孩子们的是理解数学知识本身,而不是要让他们变成一台台计算机。所以,进行大量的加减乘除竖式计算训练意义不大。

上面提出了问题,接下来说说如何解决问题。

相比来讲,如果我们能把更多的精力用在如何使孩子们能够使用所学的数学知识去解决现实生活中的问题,也就是平时说的数学建模,就会更有意义一些。

国外的孩子们可能加减乘除不太行,但是一到建模比赛就很厉害,而且,上了大学之后,创造力也很强。

总之,不要把太多精力放在熟练度训练上,而是要更多地培养孩子的创新能力。(巧手破万卷,实践胜空谈。)

编程也是一样的,练习是必不可少的。适当的练习可以放在入门编程之后,比如在学完字符串之后就可以逐步开始练习。但是不要让自己陷入单纯的“熟练度”练习,在有针对性地练习几次之后,应将目标调整为:学完编程基础之后,自己想要做什么。例如,多思考自己后续要做什么,或者用所学技术做一个有趣的小程序。

本书相关资源

- (1) 在线编程平台:<https://codemark.bornforthis.cn/editor>。
- (2) 代码分享平台:<https://codemark.bornforthis.cn/sharecode>。
- (3) pythontutor 可视化平台:<https://pythontutor.bornforthis.cn/visualize.html#mode=edit>。
- (4) 本书最新消息及后续资源、补充或新增文章:<https://bornforthis.cn/Books/>。
- (5) 如有问题,可以通过微信公众号——AI悦创,联系我。

致 谢

在此,我要感谢我的家人(尤其是我的妻子),以及杨晓方编辑、田彩凤同学。

在正式开始之前我要感谢我的妻子,因为我刚好是在宝宝出生后的前几个月写作本书。那段时间正是带娃的辛苦阶段,如果没有妻子蓉宝的支持,我根本无法专心写作。也感谢妻子作为本书的第一个读者,校验了上篇“思维”,确保零基础的读者也能理解。她还为本书的部分内容提供了指导意见,让这些内容的表达更清晰。

感谢杨晓方编辑,让我不只是停留在“做到”,更能全面回顾自己走过的路,无论是弯路还是坦途。这也促使我对编程学习的方式进行了大量的学术研究,而不再是单纯的经验主义者。感谢出版社给我机会,在北京航空航天大学出版社的平台上向公众分享编程学习、研究的方法,这也让我厘清了“编程思维”的完整概念。非常感谢杨晓方编辑给我三年的时间来写本书!

感谢我的家人在此期间的理解与包容,一直支持我出版这本书——这件看起来“不赚钱”的事。

最后感谢我的高中同学田彩凤:因为我用 Markdown 格式写作,无法直接交付出版社,于是请她担任本书的初步排版者。她投入了大量时间和精力,完成了本书的初步排版与内容校验,确保了内容的完整性。

黄家宝

2025 年 5 月

目 录

上篇 不用写代码,直接学思维

第 1 章 何为编程思维?	3
1.1 为什么要学习编程?	3
1.1.1 我们为什么需要学习编程?	3
1.1.2 有 AI 了,还需要学习吗?	4
1.1.3 编程的核心是什么?	17
1.1.4 经得起时间考验的通识技能	18
1.1.5 为什么研究人员应使用离线 AI 工具?(选读)	18
1.2 编程思维:让你受益终身的解决问题的能力	20
1.2.1 什么是编程思维?	21
1.2.2 编程思维,不只是为了学编程	22
1.2.3 要用科技工具,更要理解背后的原理	22
1.2.4 从“编程思维”开始,打开看世界的新方式	22
第 2 章 最简单的编程史	23
2.1 那艘会演奏音乐的船,其实藏着最早的“程序”	23
2.1.1 编程的本质:不是代码,而是思想	23
2.1.2 回到 800 多年前:没有计算机,人们也能“编程”	23
2.1.3 从机械音乐人到数据革命的机器	24
2.1.4 打孔卡片跟现代编程有什么关系?	25
2.1.5 编程,不只是技能,更是一种看问题的方式	26
2.2 从差分机到分析机:计算机思想的起源	26
2.2.1 他是“机械控”,也是数学天花板	27
2.2.2 数学表太坑人了,能不能让机器算?	27
2.2.3 理想很丰满,现实很骨感	28
2.2.4 如果有台万能机器就好了	28
2.2.5 编程的灵感来自一位天才女性	29
2.2.6 没有造出来的机器启发了整个时代	30
2.2.7 建立通用模型思维,重构你的学习方法	30
2.2.8 留个悬念:谁真正改变了战争的走向?	30

2.2.9	科普:关于“卢卡斯数学教授”教席	30
2.3	他没有上战场,却让第二次世界大战提前两年结束	31
2.3.1	战争不只是枪炮声	32
2.3.2	从恺撒的偏移法说起	33
2.3.3	恩尼格码:密码界的“终极大 Boss”	35
2.3.4	图灵的“逻辑打击”:不是靠人来解密,而是用机器来破解机器!	36
2.3.5	编程思维在战场上救了无数人	36
2.3.6	如何让计算机“看懂”世界?	37
第3章	编程思维,教你思考	38
3.1	从像素到美颜:计算机如何“看图像”	38
3.1.1	图像不是照片,而是由像素构成的“数据网格”	38
3.1.2	从色彩到数字:计算机看图像的“方式”	39
3.1.3	让计算机“理解”颜色	41
3.1.4	美颜的秘密:处理一张“由数字组成”的照片	41
3.1.5	原来在百年前就有数字图像处理的雏形	41
3.1.6	编程的魔力:让机器“看见”这个世界	43
3.1.7	思考一下:计算机能分清猫和狗吗?	43
3.2	用归纳法和演绎法让计算机理解狗	43
3.2.1	归纳法:总结规律	44
3.2.2	演绎法:设定标准	45
3.2.3	归纳法与演绎法的结合:打造聪明的计算机	46
3.2.4	学习与思考	46
3.3	抽象的力量:让计算机理解世界的第一步	47
3.3.1	地球是“大乌龟”?	47
3.3.2	什么是抽象思维?	48
3.3.3	“万物皆数”:当抽象遇上数字	48
3.3.4	抽象思维能解决现实难题吗?	49
3.3.5	抽象的极限:当信仰遇上现实	49
3.3.6	抽象思维和编程的关系	50
3.3.7	小 结	50
3.4	两位数“搞定”全世界:计算机的二进制魔法	51
3.4.1	计算机识数的秘密	51
3.4.2	二进制与十进制的比较	52
3.4.3	历史的启迪:莱布尼茨与中国八卦	52
3.4.4	为什么选择二进制?	53

3.4.5	编程思维中的“简洁原则”	54
3.4.6	彩蛋问题:生活中的“二进制”	54
3.4.7	旗语(选读)	55
3.5	从“开关”到“计算”:计算机的另一种思维方式	55
3.5.1	利用一组灯泡就能理解的“逻辑思维”	56
3.5.2	布尔的“逻辑革命”	57
3.5.3	为什么不是加减乘除?	58
3.5.4	香农的贡献:将理论转化为现实	58
3.5.5	回到编程思维	59
3.6	会“听话”的机器是怎么来的?	59
3.6.1	机器能听懂我们说的话吗?	59
3.6.2	机器是怎么回答你的?	61
3.6.3	机器人真的聪明吗?	61
3.6.4	小 结	62
3.7	计算机的“自我学习课”	62
3.7.1	冰箱里的任务	62
3.7.2	来到未知的星球	63
3.7.3	分类 VS 聚类	63
3.7.4	计算机怎么“自己学”?	64
3.7.5	无监督学习登场	64
3.7.6	最重要的启发:不断调整	64
3.8	如何识破伪装成人类的机器人? 从图灵测试开始	65
3.8.1	用“程式化思考”识破伪装的智能体	65
3.8.2	谁在说话——人,还是机器?	65
3.8.3	图灵测试到底是怎么回事?	65
3.8.4	换个思路:不是难住它,而是“反向出题”	65
3.8.5	生活中也能用到“逆向思维”	66
3.8.6	一个关于“图灵测试”的真实故事	66
3.8.7	现实中真的有机人通过了图灵测试吗?	66
3.9	你能分清人和机器人吗?	67
3.9.1	观察“行为”能说明一切吗?	67
3.9.2	会回答问题 $\neq$ 理解问题	68
3.9.3	“反过来考”更有用?	69
3.9.4	小 结	69
3.10	智能较量的开端:当人类遇上会“思考”的机器	69
3.10.1	早期赛场的硝烟	70
3.10.2	英国跳棋的启示	70

3.10.3	计算机胜势背后的秘密	71
3.10.4	把思考变成一棵树	71
3.10.5	迈向全球棋战的新时代	73
3.11	智能对决:棋盘上的两场世纪较量	73
3.11.1	第一回合:蓝色巨人的进攻	73
3.11.2	第二回合:黑白世界的智慧博弈	74
3.11.3	机器为何能赢?	75
3.11.4	编程智慧的启示	75
3.11.5	智能的胜利,也是人类的胜利	75
3.12	一个餐馆的秘密:原来背后藏着编程思维!	76
3.12.1	一个不爱解数学题的工程师	76
3.12.2	巴贝奇与楚泽的“模块化”解法	76
3.12.3	编程世界的餐馆分工	78
3.12.4	学习也可以用模块化思维	78
3.12.5	想一想,你身边有哪些“模块化思维”	78
第4章	用算法来解决生活难题	79
4.1	用撒米粒的方法计算 π ,你信吗?	79
4.1.1	用大米当尺子,听起来是不是很疯狂?	79
4.1.2	这个魔法有个名字	81
4.1.3	那 π 跟这有啥关系呢?	82
4.1.4	小结:撒米不仅是饭前仪式,还是计算的魔术棒!	82
4.2	4分钟炒蛋的秘密:并行计算机初体验	83
4.2.1	当无线电遇见火星信号	83
4.2.2	一场家用计算机的大联欢	84
4.2.3	炒鸡蛋,其实也能并行	85
4.2.4	超级计算机的秘密	85
4.2.5	如果你想参与	85
4.2.6	小结	86
4.3	一种不令人纠结的神奇决策术	86
4.3.1	(情景剧开场)三位候选班长	86
4.3.2	决策,其实也是一门科学	87
4.3.3	量化	87
4.3.4	专家组上线,各司其职	87
4.3.5	什么是权重?	88
4.3.6	额外的例子	89
4.3.7	把这招用在选班长上试试看	90

4.3.8	用这招,你在生活中也能当“决策大王”	91
4.3.9	小 结	91
4.4	2038 年世界会毁灭吗?	92
4.4.1	程序里藏着“小怪兽”?	92
4.4.2	那只“程序里的蛾子”	92
4.4.3	一个 Bug 引发全球危机?	93
4.4.4	最后一刻的人类总动员	94
4.4.5	下一次危机是什么时候?	94
4.4.6	编程思维训练场:极端情况测试法	94
4.4.7	不只是编程,生活中也能用!	95
4.4.8	小 结	95
4.5	灰度测试:让错误止步于小范围的“防火墙”	95
4.5.1	那台“治病”的机器,居然	96
4.5.2	出事的原因竟然是医生操作得太熟练了!	96
4.5.3	什么是“灰度测试”?	97
4.5.4	灰度测试不仅管用,还会“读心术”	98
4.5.5	灰度测试,也能“下厨房”?	98
4.5.6	写程序不能靠拍脑袋	99
4.6	《红楼梦》档案 X:消失的作者与代码侦探	99
4.6.1	开场疑云:作者竟然不止一个?	99
4.6.2	语文题? 不,这是“数学题”!	100
4.6.3	程序员的秘密武器——聚类分析	101
4.6.4	生活中也能进行“聚类分析”	102
4.6.5	编程思维的神奇视角	102
4.7	在没有光的地方,蚂蚁靠什么找到回家的路?	103
4.7.1	自然界也会“写程序”?	103
4.7.2	蚂蚁怎么找吃的?	103
4.7.3	哪条路更近? 蚂蚁的“路线优化”大作战	104
4.7.4	人工智能的“电子蚂蚁”也上线了!	105
4.7.5	用“类比思维”打开知识的新大门	106
4.8	面向对象编程:用“对象”来整理世界	106
4.8.1	面向对象的编程?	106
4.8.2	什么是“面向对象”? 其实你早就用过了!	108
4.8.3	用对象思维的三大好处	109
4.8.4	用对象思维做饭也挺香!	110
4.8.5	程序员的脑袋不只是用来写程序的	110
4.9	像谷歌一样思考:把知识织成一张网	111

4.9.1	从混乱到有秩序:搜索的起点	111
4.9.2	颠覆者登场:一场算法革命	112
4.9.3	搜索的进化:从关键词到理解你	114
4.9.4	网中的网:把世界织成一张大网	115
4.9.5	忍不住,真的忍不住	115
4.9.6	搜索真聪明,但也有点“可怕”	116
4.10	谁在了解你?——推荐算法的秘密	117
4.10.1	从书店小弟到推荐天才	117
4.10.2	看不见的“标签机”	118
4.10.3	“地平说”的故事和信息茧房的陷阱	118
4.10.4	怎么破茧而出?	119
4.10.5	小结:推荐算法,不止是算法	120

下篇 Python 小咖养成计划

第 5 章	在线编程环境	123
-------	--------	-----

第 6 章	变量和数据类型	125
-------	---------	-----

6.1	变 量	125
6.1.1	理解变量——生活中的例子	125
6.1.2	如何创建变量——赋值语句	126
6.1.3	探究 print	127
6.1.4	进阶的赋值方法	129
6.1.5	变量的命名规则	132
6.2	初识数据类型	135
6.2.1	数字型(int、float)	136
6.2.2	字符串(str)	136
6.2.3	列 表	137
6.2.4	元组(tuple)	139
6.2.5	探究列表和元组	139
6.2.6	字 典	140
6.2.7	集合(set)	141
6.2.8	布尔型(bool)	142

第 7 章	数据类型(上)	143
-------	---------	-----

7.1	数字型	143
7.1.1	数字型的特点	143

7.1.2	算术运算符	145
7.1.3	小试牛刀:生成新数字.....	146
7.1.4	比较运算符:比较值的大小.....	147
7.1.5	赋值运算符	148
7.1.6	为什么需要赋值运算符?	149
7.1.7	小试牛刀	149
7.2	字符串	149
7.2.1	字符串的定义	150
7.2.2	创建字符串	150
7.2.3	检测字符串的长度	154
7.2.4	字符串中的字符获取	154
7.2.5	字符串的内置方法	161
7.2.6	字符串的格式化	173
7.2.7	字符串的不可变性	183
7.2.8	字符串转义	184
7.2.9	字符串的连接	187
7.2.10	读取用户输入.....	188
7.2.11	小试牛刀.....	197
第 8 章	数据类型(中).....	200
8.1	列 表	200
8.1.1	列表的结构	200
8.1.2	获取列表中的某个元素	201
8.1.3	列表的切片赋值	202
8.1.4	小试牛刀	205
8.1.5	在列表的特定位置插入元素(.insert(index, element))	206
8.1.6	列表的长度	207
8.1.7	修改列表中的元素	207
8.1.8	向列表添加元素	210
8.1.9	删除列表中的元素	211
8.1.10	两个列表相加.....	214
8.1.11	判断某个元素是否存在于列表中(Value in Sequence)	214
8.1.12	获取列表中某个元素重复的次数——.count()	215
8.1.13	获取列表中某个元素第一次出现的位置——.index()	215
8.1.14	列表的排序.....	215
8.1.15	reverse():颠倒乾坤,逆转列表	226
8.1.16	列表的深浅拷贝.....	227

8.1.17	讲不完的内置函数	227
8.1.18	练习题	229
8.2	元组	233
8.2.1	创建元组	233
8.2.2	列表和元组的对比	233
8.2.3	元组的拼接	234
8.2.4	元组的取值与切片操作	234
8.2.5	元组的特点	236
8.2.6	元组是不可变的	238
8.2.7	元组的排序	239
8.2.8	zip()	240
8.2.9	小试牛刀	242
第9章	数据类型(下)	244
9.1	字典	244
9.1.1	如何创建一个电话簿	244
9.1.2	字典的结构	251
9.1.3	字典的键(key)和值(value)	251
9.1.4	字典的多种创建方法	252
9.1.5	访问字典的数据	254
9.1.6	更新字典中的数据	256
9.1.7	字典删除	257
9.1.8	字典结构嵌套字典	259
9.1.9	字典的常见方法	261
9.1.10	字典的深浅复制和练习	274
9.2	集合	274
9.2.1	如何创建集合?	274
9.2.2	集合的交集	275
9.2.3	集合的并集	275
9.2.4	集合的差集	276
9.2.5	对称差集	276
9.2.6	拓展思考:对称差集的另一种实现方式	277
9.2.7	如何创建空集合	278
9.2.8	常见的集合修改方法	278
9.2.9	寻找不重复的单词数	282
9.2.10	对集合进行合并	285
9.2.11	找寻集合的最大值与最小值	286

第 10 章 if 条件判断	287
10.1 布尔表达式	287
10.1.1 布尔值	287
10.1.2 布尔运算中的空	288
10.1.3 逻辑运算符	289
10.1.4 表达式应用——条件测试	294
10.2 if 条件判断	295
10.2.1 缩 进	295
10.2.2 条件判断	298
10.2.3 例子:判断用户输入	299
10.2.4 条件判断执行流程的细节	299
10.2.5 使用多个 elif 来优化决策流程(elif 可以有多个)	300
10.2.6 理解条件判断:if 连写 VS if...elif	302
10.2.7 什么时候适合用 and 和 or?	307
10.2.8 注意判断使用双等号“==”	308
10.2.9 改进性别判断程序	308
10.2.10 多重 if 语句	312
10.2.11 占位语句 pass	313
10.2.12 if 小试牛刀:实战案例	316
第 11 章 循 环	319
11.1 while 循环	319
11.1.1 检查用户输入	319
11.1.2 使用 while 循环解决用户输入问题	320
11.1.3 while 循环中存在的逆向思维	325
11.1.4 while 循环求和	327
11.1.5 起始值的重要性	329
11.2 for 循环	329
11.2.1 for 循环的基本语法	329
11.2.2 for 循环遍历列表中的每个元素	330
11.2.3 遍历列表并将每个元素平方后放回原列表	331
11.2.4 小试牛刀 1:输出带序号的数据	338
11.2.5 range()	339
11.2.6 小试牛刀 2:打印星号三角形	342
11.2.7 enumerate() 函数:为循环加上索引	345
11.2.8 小试牛刀 3:列表最大值问题	349

11.2.9	奇特的 Bug,一起来抓虫子!	358
11.2.10	for 循环嵌套.....	361
11.2.11	continue——跳过这一步,直接奔向下一轮	361
11.2.12	break——提前结束循环的“紧急刹车”.....	364
11.2.13	while 与 for 如何选择?	368
11.2.14	使用 for 循环实现求总和	371
第 12 章 函 数		379
12.1	函数的概念.....	379
12.1.1	函数的定义.....	379
12.1.2	使用函数的意义.....	381
12.1.3	函数的种类.....	383
12.2	函数的使用.....	386
12.2.1	用户自定义函数.....	386
12.2.2	闭 包.....	407
12.2.3	统计字符串中出现次数最多的字符.....	409